

# Hospital Management System

Object-Oriented Programming Project

Aadil Jawad Husain BSCS25127



Elegant Hospital Elevation  
Design Ideas for India

# Why This Project?

---

## The Problem

Hospitals deal with massive amounts of data—patients, doctors, appointments, and bills. Managing this manually is prone to errors and data loss.

## The Solution

A C++ based application using Object-Oriented Principles to automate hospital operations, ensure data persistence, and provide role-based access.

# System Architecture



## Unified Design

Our system is built on a clear class hierarchy. We have a central "User" base class that branches out into specific hospital roles.

- **Base Class:** User (Abstract)
- **Derived Classes:** Admin, Doctor, Patient
- **Support Entities:** Appointment, Bill
- **Persistence:** File Handling & Serialization

# The "User" Base Class

---



## Attributes

Protected data: UserID, Name, Password, and Role. This ensures derived classes can access core data easily.



## Pure Virtuals

Includes `showMenu() = 0`. Every user *must* have a menu, but the logic depends on who they are.



## Serialization

Virtual `serialize()` and `deserialize()` methods allow us to convert objects into text lines for storage.

# Inheritance in Action

---

| Subclass | Extra Attributes    | Specific Responsibility                          |
|----------|---------------------|--------------------------------------------------|
| Admin    | AdminLevel, Dept    | Managing doctors, patients, and global reports.  |
| Doctor   | Specialization, Fee | Managing appointments and patient prescriptions. |
| Patient  | Age, History, Bill  | Booking slots and viewing medical records.       |

# Admin: The Controller

---

## Credentials & Control

Admins are the backbone of the system infrastructure. They have unique ADM-XXXX credentials to manage the roster.

**Seeding Check:** On first boot, the system automatically creates a Master Admin if the database is empty, preventing a logout.

# Clinical Interaction

---



## Booking

Patients can view available doctors and book appointments which are marked as "Pending".



## Triage

Doctors log in to view their specific schedule and mark appointments as "Completed" after checkups.



## Prescriptions

Physicians append diagnostic notes directly to the patient's medical history attribute.

# Supporting Entities

---

## **Appointment Class**

Binds Doctor ID and Patient ID. Tracks date, time, and current status (Pending/Confirmed/Done).

## **Billing Logic**

Automatically calculates dues based on consultation fees. Admins can override or waive these for financial management.



# Key OOP Pillars Used

---



## Encapsulation

Attributes are private/protected.  
We use getters/setters to protect sensitive patient info.



## Polymorphism

Using User\* currentUser pointer to launch the correct dashboard menu at runtime.



## Abstraction

The "User" class hides complexity.  
The main program just calls methods on a User pointer.

# File Handling

## **Data Persistence**

We use `fstream` to save data to `.txt` files. This ensures your data isn't lost when the program closes.

**Serialization:** Every object knows how to turn its variables into a comma-separated line (CSV style) for easy reading/writing.

# Storage Template

---

## GENERIC STORAGE

### Reusable Code

Instead of writing separate logic to store Doctors, Patients, and Admins, we created a **Template Class**.

This Storage class handles adding, removing, and finding items for any data type we throw at it, making our code clean and efficient.

# The Execution Loop

---

